Supplementary Material, Centrality-Based KV-Cache Eviction

Companion to the main report (`REPORT.pdf`): detailed per-press systems figures and per-task result tables. See the report §4.4–4.5 for the summary and interpretation.

S.1 Per-press systems figures

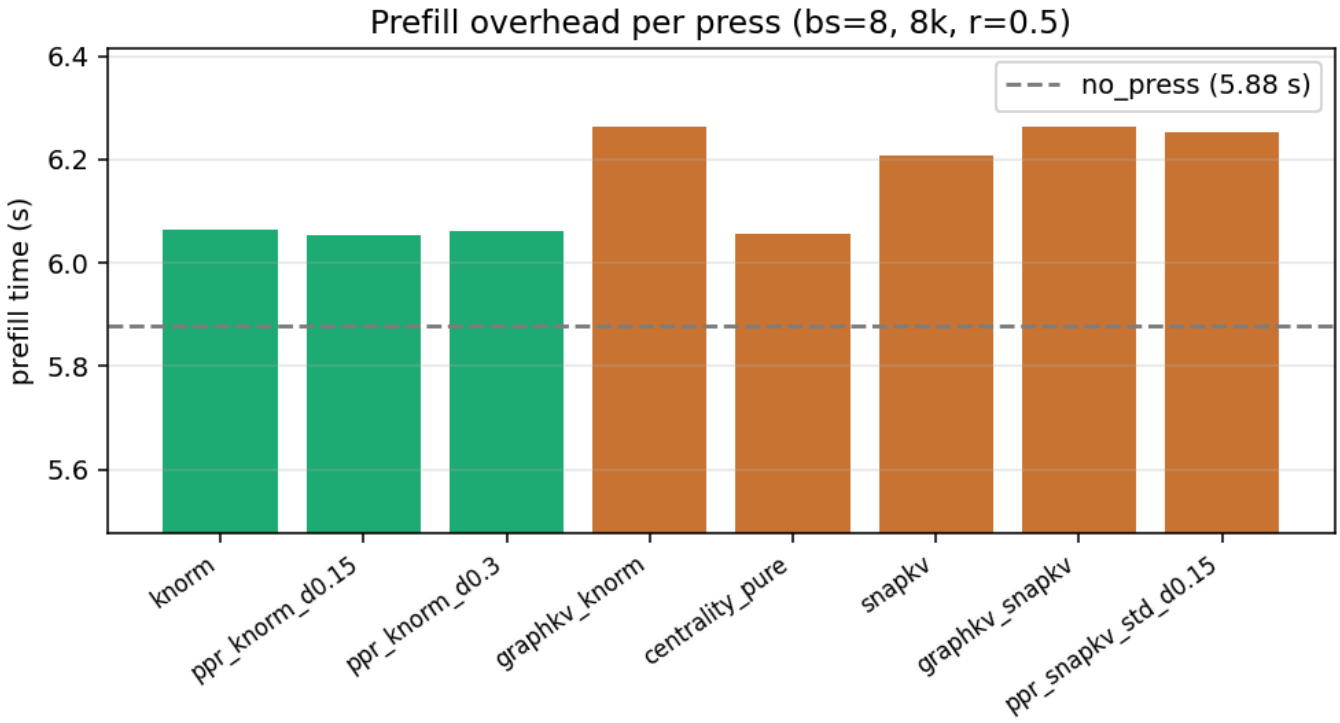

Figure S1: Prefill scoring overhead, the only press-dependent systems cost; `ppr_knorm` ≈ `knorm`, cheaper than `graphkv` / SnapKV-based presses.

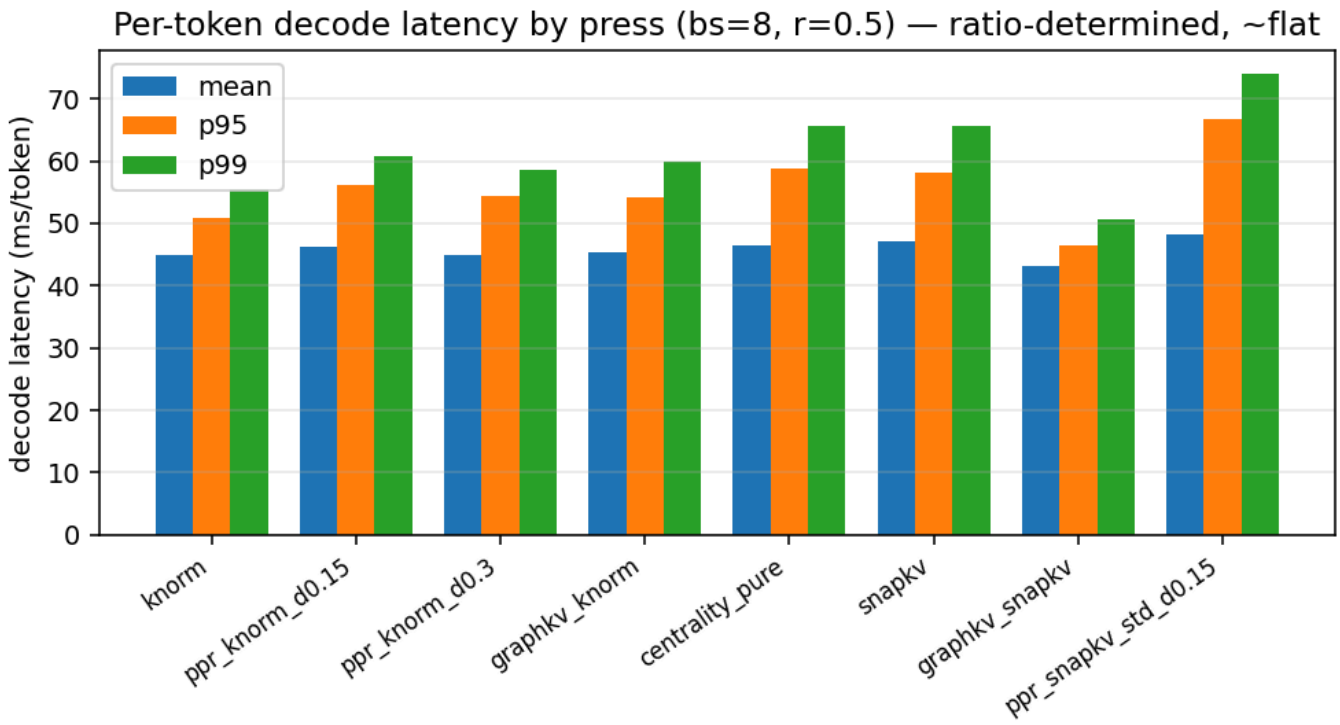

Figure S2: Per-token decode latency (mean/p95/p99) by press at r=0.5, ratio-determined, ~flat.

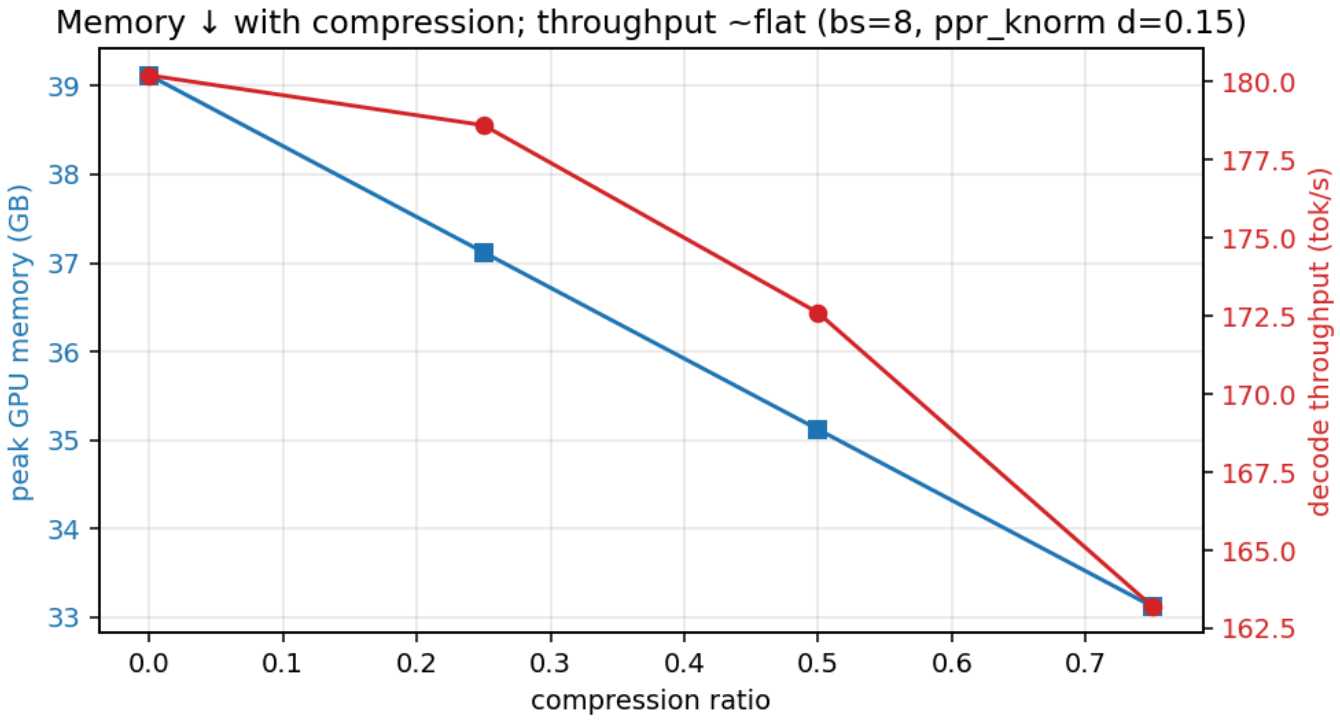

Figure S3: Peak GPU memory falls linearly with compression; decode throughput ~flat at this scale.

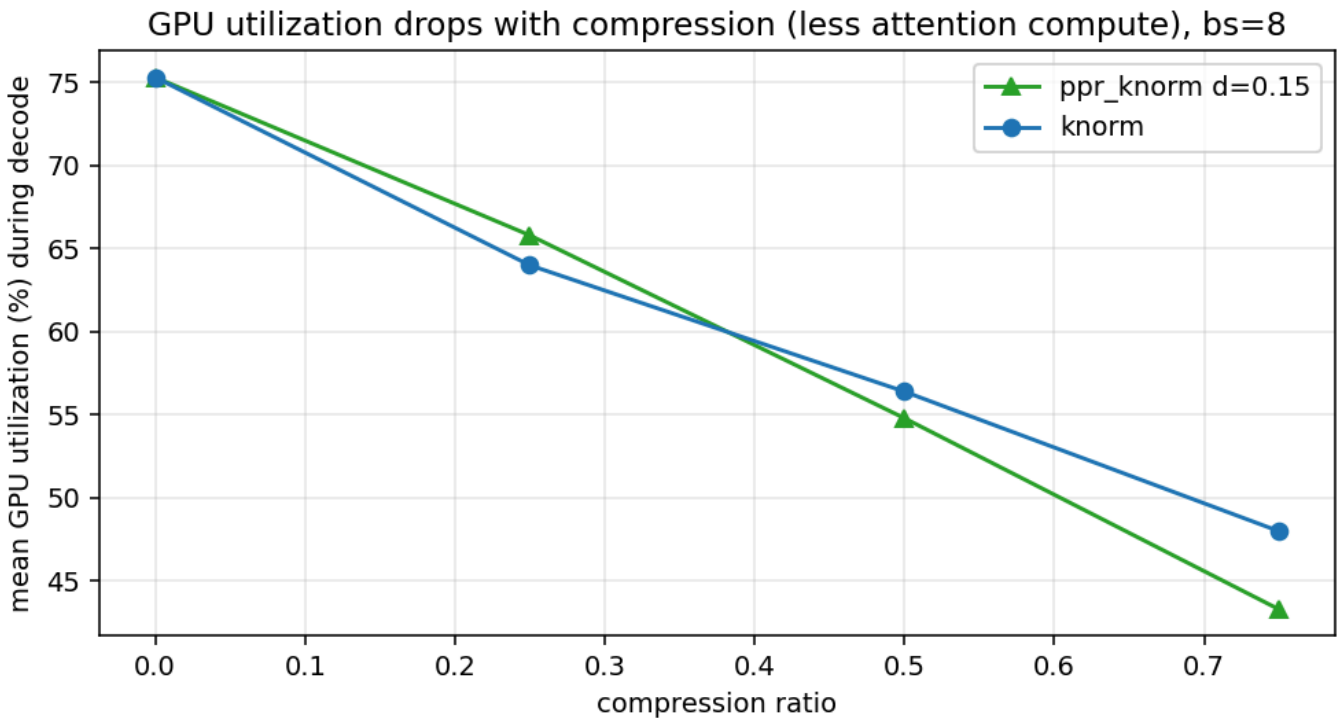

Figure S4: Mean decode GPU utilization drops with compression (less attention compute per token).

S.2 Per-task tables

RULER 4096 @ compression 0.5 (string-match %, usable tasks)

task	no_press	knorm	graphkv	snapkv	ppr d=0.15	pure
niah_single_1	100	100	100	94.7	100	0
niah_single_2	100	80.0	20.0	97.1	100	0
niah_single_3	100	2.9	5.9	0	73.5	0
niah_multikey_1	100	30.0	40.0	90.0	90.0	6.7
niah_multikey_2	100	4.8	81.0	61.9	100	0
niah_multikey_3	100	4.5	36.4	22.7	18.2	0
qa_2	58.1	32.3	48.4	48.4	48.4	29.0

LongBench multi-hop @ compression 0.5 (F1 %)

subtask	no_press	knorm	graphkv	snapkv	ppr d=0.15
hotpotqa	58.5	54.1	45.9	56.7	49.2
2wikimqa	53.1	45.9	46.7	53.5	53.4
musique	30.7	19.5	30.9	25.2	23.4

Per-run configs + metrics are under `project/reproduction/results/` (`config.yaml` + `metrics.json` each; the aggregated per-example `*_long.csv` are report-only, regenerable, see `project/reproduction/README.md`). Regenerate figures with `analysis/make_figures.py` and paired stats with `analysis/analyze_paired.py`.

S.3 Iso-accuracy operating-point curve (16k, batch 8)

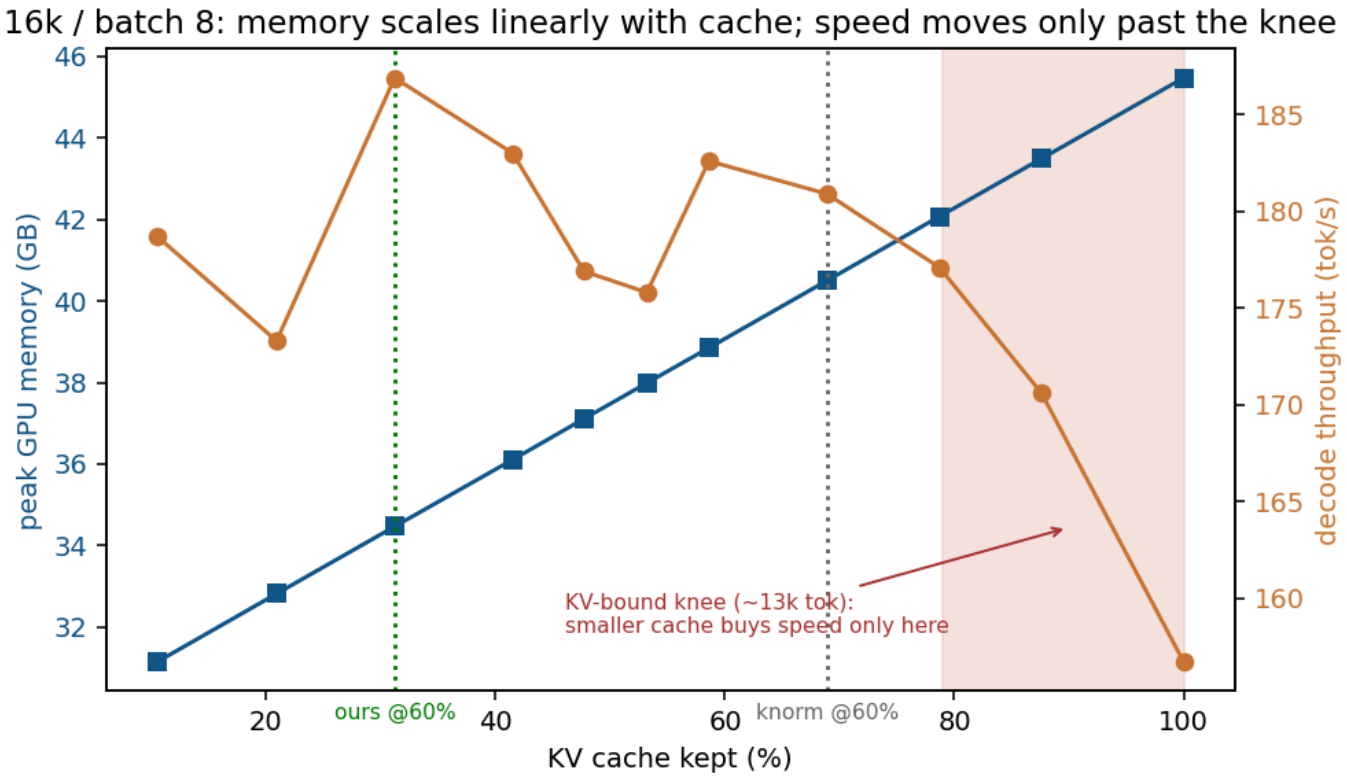

Figure S5: Directly measured peak GPU memory and decode throughput vs. kept-cache fraction (16k prompt, batch 8, `analysis/iso_systems.py` on `results/results_iso_systems_16k_bs8.csv`). Peak memory falls linearly with the cache; throughput is essentially flat once the cache is compressed (decode is weight-bound at 8B/16k/bs8). The dotted lines mark the iso-accuracy operating points for a 60 % RULER target: the base press must keep ~69 % of the cache, ours only ~31 %, same accuracy, 2.2× less cache and ~6 GB less peak memory, while throughput is unchanged (the speed win needs a more KV-bound regime).

Appendix, Code & data artifacts

- **Press code:** `kvpress/presses/centrality_press.py`; registration in `kvpress/__init__.py`, `evaluation/evaluate_registry.py`, `tests/default_presses.py`, `README.md`; unit test in `tests/presses/test_centrality_press.py`.
- **GraphKV comparison baseline (`project/additional_benchmarks/`):** `decay_propagation_press.py`, `test_decay_propagation_press.py`, `run.py` (runtime-injected registry entry), `README.md`.
- **Benchmark + analysis (`project/reproduction/`):** `scripts/sweep_longbench.py`, `scripts/submission_grid_llama.sh`, `scripts/fetch_eval_datasets.sh`; `analysis/analyze_paired.py`, `analysis/make_figures.py`, `analysis/speed_memory.py`, `analysis/iso_systems.py`, `analysis/recall_probe.py`, `analysis/rank_vs_board.py`, `analysis/kvpress_leaderboard_raw.csv`.
- **Data/logs (`project/reproduction/results/`):** `board_grid/` (with `predictions.csv`) and `ruler_screen/` run directories (each with `config.yaml` + `metrics.json`); figures in `report/figures/` (`fig1`–`fig12`).
- **Reproducibility:** `requirements.txt`, `Dockerfile`, `README.md` (how-to-benchmark + traceability), `scripts/benchmark.sh` (one-command benchmark), `.github/workflows/centrality-ci.yml` (CI: comparison-benchmark tests on every push).